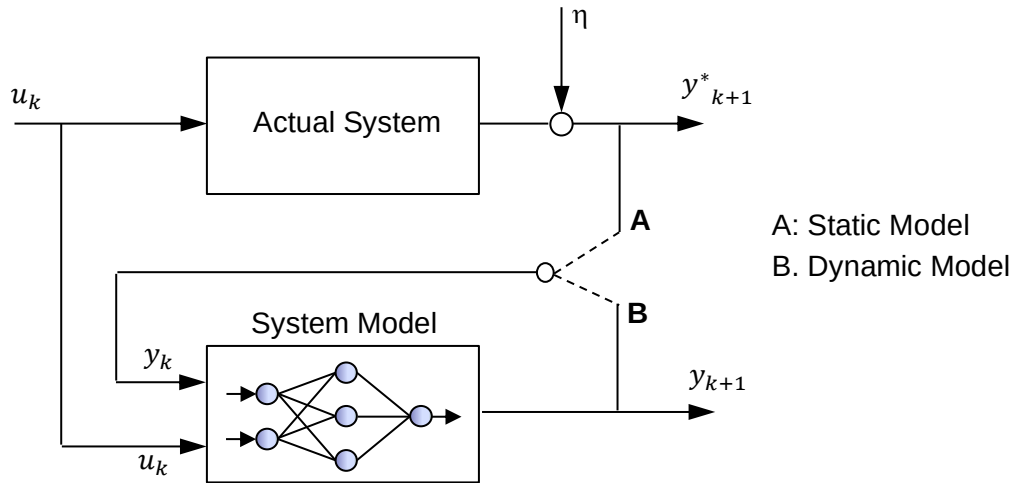


3. Modeling of Dynamic Systems

The input-output mapping capabilities of neural networks can be used for constructing models of linear and nonlinear dynamic systems. Given a set of system input-output data, the neural network can be trained to perform as a model of the system which can be used to simulate and analyze the response of the system under diverse operating conditions different to those used when training the neural network.

Both static and recurrent neural networks can be used for modeling systems, each one with particular structure and characteristics. Figure 1 shows the general structure of the modeling process. The actual system has input u_k and output y_{k+1}^* which are available for modeling. This training data is supposed to be rich enough comprising the complete dynamics of the system. The systems model has input u_k and output y_{k+1} . The objective is to train the neural network so that the difference between the model output y_{k+1} and the actual system output (desired output) y_{k+1}^* is minimized. However, depending on the precedence of the model input y_k the system model could be a static model or a dynamic model.

As it is shown in Figure 1, when the model input y_k comes from the actual system (measured output), as it is represented by point A, the model turns to be a static model. However, when the model input y_k comes from the output of the model itself, as it is represented by point B, the model turns to be a dynamic model.



The equation described by the static model is:

$$y_{k+1} = f(y_k^*, u_k) \quad (1)$$

whereas the equation described by the dynamic model is:

$$y_{k+1} = f(y_k, u_k) \quad (2)$$

From equation (1) it is noted that the model output does not become input in the next step and, therefore, it is a static system. However, in equation (2) it is noted that the model output is fed back to become input in the next step and, therefore, it is a truly dynamic model. From Figure 1, two points are highlighted:

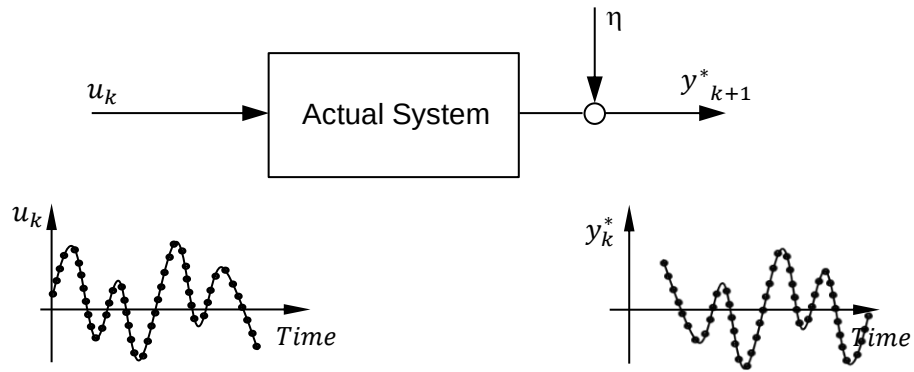
- The static model will require the actual system be in operation for providing the input y_k^* . This fact is not a concern when the model is used in real time (when controlling the system, for

instance), but it may be inconvenient when the model is used for simulating and predicting the response of the systems.

This is not the case of the dynamic system since the input y_k^* is generated by the same recurrent network

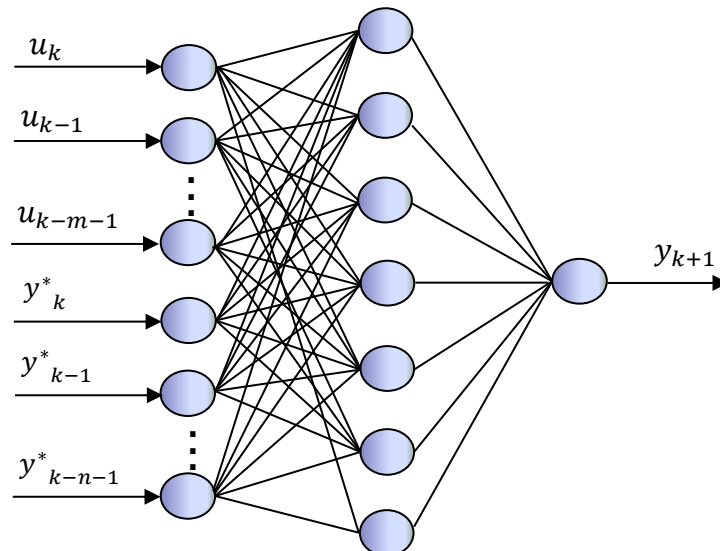
- It is expected the dynamic model be more robust against measurement noise η , since the noisy measured output is used for training the network only, but it is not used as network input. After trained, the dynamic network is completely independent of the measured noisy output y_k^* . However, the static network will always be affected by the measurement noise since one of its inputs is the measured output signal y_k^*

The richness of the training data is an important factor for constructing good meaningful models. For this, the input data u_k should be rich and varied for exciting all the dynamics of the systems. Figure 2 shows an actual system with its input and measured output data which may be contaminated with noise η .



3.1. Modeling with Static Networks

When using static networks for modeling dynamic systems, it is necessary to create an *artificial dynamics* by incorporating past system inputs u_k and past outputs y_k^* as neural network inputs as it shown in Figure 3 where it is noted that y_{k+1} depends on m past values of u_k and n past values of y_k^*



The equation describing the neural network of Figure 3 is:

$$y_{k+1} = f(y_k^*, y_{k-1}^*, \dots, y_{k-n-1}^*, u_k, u_{k-1}, \dots, u_{k-m-1}) \quad (3)$$

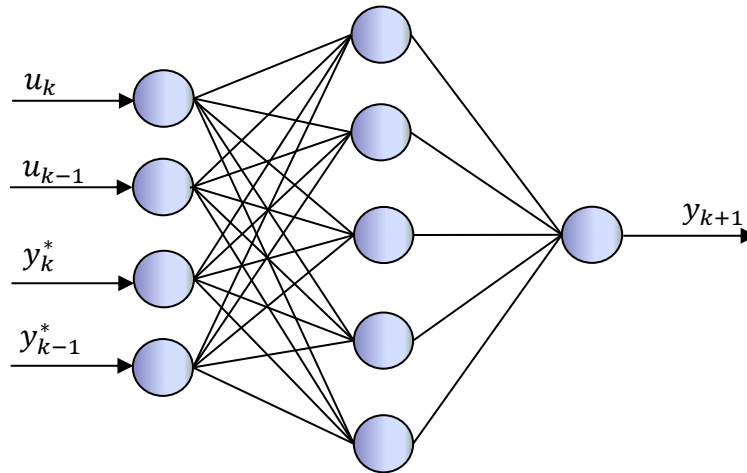
Since the output y_{k+1} is not fed back to the input of the network, it is clear that this is a static network that can be trained with the standard error back propagation algorithm.

The number of m past values of u_k and n past values of y_k^* can be determined by trial and error by training and validating the neural network. However, the values of m and n can be determined by analyzing the approximated linear model of the system. For instance, if the linearized model is represented by the transfer function $H(z)$ and the corresponding difference equation given by equation (4) and (5):

$$H(z) = \frac{Y(z)}{U(z)} = \frac{b_1 z + b_0}{z^2 + a_1 z + a_0} \quad (4)$$

$$y_{k+1} = -a_1 y_k - a_0 y_{k-1} + b_1 u_k + b_0 u_{k-1} \quad (5)$$

then, a first trial for defining the inputs of the static neural network could be: $y_k, y_{k-1}, u_k, u_{k-1}$. Figure 4 shows the corresponding neural network.

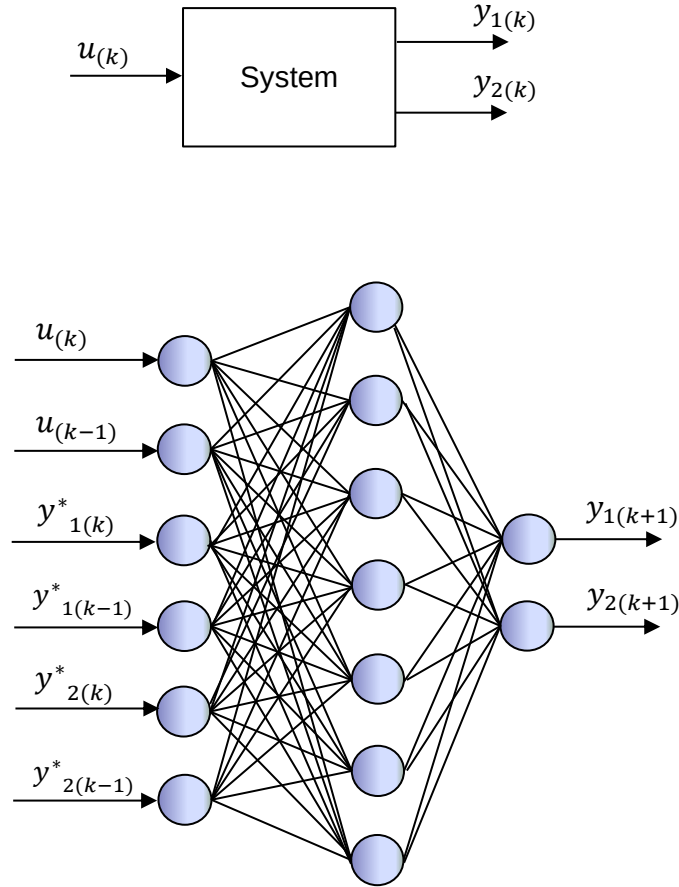


For training this neural network, the input-output data presented in Figure 2 must be arranged as it is shown in Figure 5 where input and output training data is ready to be presented to the network.

Training Data				
u_k	u_{k-1}	y_k^*	y_{k-1}^*	y_{k+1}^*
u_0	u_{-1}	y_0^*	y_{-1}^*	y_1^*
u_1	u_0	y_1^*	y_0^*	y_2^*
u_2	u_1	y_2^*	y_1^*	y_3^*
...	...	...	...	...
u_{N-1}	u_{N-2}	y_{N-1}^*	y_{N-2}^*	y_N^*

In most cases a bias neuron is not required at the input of the network. A bias neuron with constant input is equivalent to have a constant term in the equation of the dynamical system. If it is the case an input bias neuron can be considered.

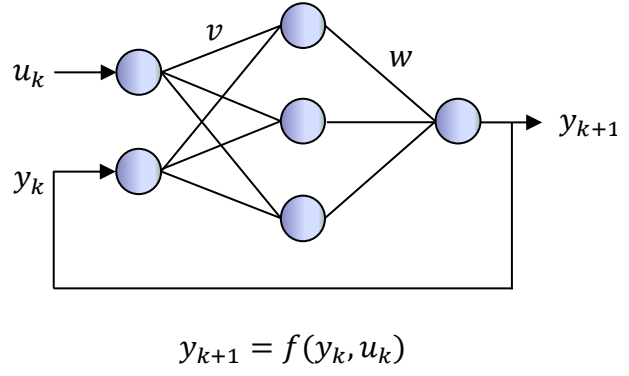
A similar approach can be applied to systems with several inputs and several outputs: the network inputs comprise the past values of each system input $u_{1(k)}$, $u_{2(k)}$, etc., as well as the past values of each system output $y_{1(k)}^*$, $y_{2(k)}^*$, etc. For the system with one input and two outputs shown in Figure 6, one possible static neural network model is shown in Figure 7.



3.2. Modeling with Dynamic Networks

Recurrent neural networks can be directly applied for the modeling of dynamic systems. For the actual system shown in Figure 2, the dynamic neural network model is shown in Figure 8 where it is noted that the output becomes input in the next time step. The system could be SISO or MIMO, in both cases the training process is the same. In the case of MIMO systems, u_k and y_k are vector signals.

Data	
u_k	y_k^*
u_0	y_0^*
u_1	y_1^*
u_2	y_2^*
u_3	y_3^*
...	...
u_N	y_N^*



For a recurrent network with several outputs, the output vector is $y_k = [y_{1(k)} \ y_{2(k)} \ \dots \ y_{n(k)}]^T$ and the desired output vector is $y_k^* = [y_{1(k)}^* \ y_{2(k)}^* \ \dots \ y_{n(k)}^*]^T$ where suffix n represents the size of vector y_k . The training objective is to find the weighting coefficients v and w minimizing the cost function:

$$J = 0.5(y_1 - y_1^*)^T (y_1 - y_1^*) + 0.5(y_2 - y_2^*)^T (y_2 - y_2^*) + \dots + 0.5(y_N - y_N^*)^T (y_N - y_N^*)$$

$$J = \frac{1}{2} \sum_{k=1}^N (y_k - y_k^*)^T (y_k - y_k^*)$$
(7)

where y_k and y_k^* are the vector network output signal and the desired vector output signal, and N represents the size of the available data for training.

The weighting coefficients v and w are updated using the gradient descent method as follows:

$$v = v - \eta \frac{\partial J}{\partial v}$$

$$w = w - \eta \frac{\partial J}{\partial w}$$
(8)

where $\overline{\partial J / \partial v}$ and $\overline{\partial J / \partial w}$ represent the total partial derivatives of cost function J respect to weighting coefficients v and w , respectively. These equations are applied to each weighting coefficient v_{ij} and w_{jk} .

The dynamic neural network can be trained using the Back Propagation Through Time BPTT algorithm or the Dynamic Back Propagation DBP algorithm. In the following, the DBP algorithm will be used because it is more efficient in terms of memory requirements.

Computing the total partial derivatives of costs function J respect to weighting coefficients v and w :

$$\frac{\partial J}{\partial v} = (y_1 - y_1^*)^T \frac{\partial y_1}{\partial v} + (y_2 - y_2^*)^T \frac{\partial y_2}{\partial v} \dots \dots + (y_N - y_N^*)^T \frac{\partial y_N}{\partial v}$$
(9)

$$\frac{\partial J}{\partial w} = (y_1 - y_1^*)^T \frac{\partial y_1}{\partial w} + (y_2 - y_2^*)^T \frac{\partial y_2}{\partial w} \dots \dots + (y_N - y_N^*)^T \frac{\partial y_N}{\partial w}$$

Total partial derivatives:

$$\frac{\partial y_1}{\partial v}, \frac{\partial y_2}{\partial v}, \dots \dots, \frac{\partial y_N}{\partial v}$$

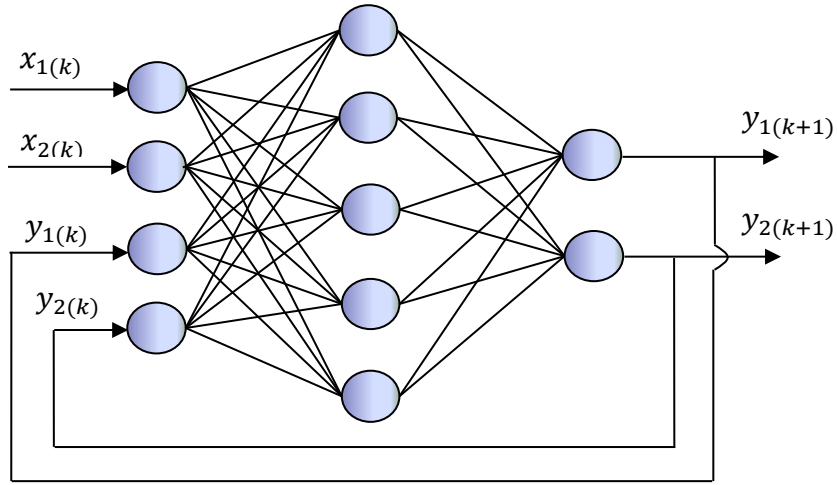
$$\frac{\partial y_1}{\partial w}, \frac{\partial y_2}{\partial w}, \dots \dots, \frac{\partial y_N}{\partial w}$$

are computed using the recursive expression:

$$\frac{\partial y_{(k+1)}}{\partial w} = \frac{\partial y_{(k+1)}}{\partial w} + \frac{\partial y_{(k+1)}}{\partial y_{(k)}} \frac{\partial y_{(k)}}{\partial w}$$

(10)

For the recurrent network with two outputs shown in Figure 9:



$$y_{k+1} = f(y_k, x_k) \quad y_k = \begin{bmatrix} y_{1(k)} \\ y_{2(k)} \end{bmatrix} \quad x_k = \begin{bmatrix} x_{1(k)} \\ x_{2(k)} \end{bmatrix}$$

The recursive equation for computing total partial derivatives:

$$\frac{\partial y_{(k)}}{\partial w} = \begin{bmatrix} \frac{\partial y_{1(k)}}{\partial w} \\ \frac{\partial y_{2(k)}}{\partial w} \end{bmatrix}$$

(11)

Jacobian matrix:

$$\frac{\partial y_{(k+1)}}{\partial y_{(k)}} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{1(k+1)}}{\partial y_{2(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{2(k+1)}}{\partial y_{2(k)}} \end{bmatrix} \quad (12)$$

$$\frac{\partial y_{(k+1)}}{\partial w} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial w} \\ \frac{\partial y_{2(k+1)}}{\partial w} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial w} \\ \frac{\partial y_{2(k+1)}}{\partial w} \end{bmatrix} + \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{1(k+1)}}{\partial y_{2(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{2(k+1)}}{\partial y_{2(k)}} \end{bmatrix} \begin{bmatrix} \frac{\partial y_{1(k)}}{\partial w} \\ \frac{\partial y_{2(k)}}{\partial w} \end{bmatrix} \quad (13)$$

$$\frac{\partial y_{(k+1)}}{\partial v} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial v} \\ \frac{\partial y_{2(k+1)}}{\partial v} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial v} \\ \frac{\partial y_{2(k+1)}}{\partial v} \end{bmatrix} + \begin{bmatrix} \frac{\partial y_{1(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{1(k+1)}}{\partial y_{2(k)}} \\ \frac{\partial y_{2(k+1)}}{\partial y_{1(k)}} & \frac{\partial y_{2(k+1)}}{\partial y_{2(k)}} \end{bmatrix} \begin{bmatrix} \frac{\partial y_{1(k)}}{\partial v} \\ \frac{\partial y_{2(k)}}{\partial v} \end{bmatrix} \quad (14)$$

Simple partial derivatives of network outputs respect to weighting coefficients v and w :

$$\partial y_{1(k+1)}/\partial w, \partial y_{2(k+1)}/\partial w, \partial y_{1(k+1)}/\partial v, \partial y_{2(k+1)}/\partial v$$

as well as the elements of the Jacobian matrix:

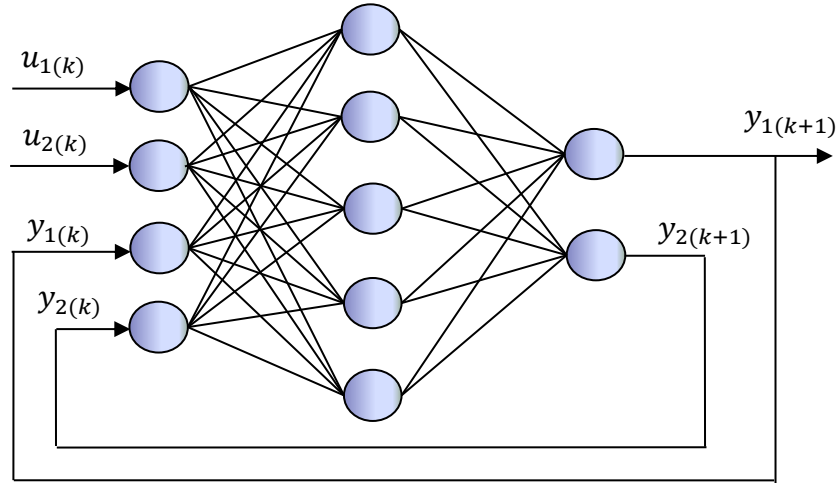
$$\partial y_{1(k+1)}/\partial y_{1(k)}, \partial y_{1(k+1)}/\partial y_{2(k)}, \partial y_{2(k+1)}/\partial y_{1(k)}, \partial y_{2(k+1)}/\partial y_{2(k)}$$

can be directly computed from the neural network in the last time step, since all the equations of the neural network for computing output from inputs are known.

The network output vector y_k can also be considered as the system state vector whose components (state variables) can be completely or partially measured. The network presented in Figure 9 corresponds to the case where all state variables are available for network training. However, in many situations not all state variables are available and the dynamic neural network model must be trained using available data only. Two approaches can be considered for constructing the model:

- Consider the state vector comprising the available state variables only. In this case, the network is trained using the same method presented before
- Consider the state vector comprising both available and non-available state variables but the network training error is defined considering the available variables only. The total partial derivatives of the state vector respect to weighting coefficients $\partial y_k/\partial v$ and $\partial y_k/\partial w$ are computed using the recursive expression given by equation (10). For the case of linear systems, it is known that a minimal realization state-space model can be constructed using the input and output data. A minimal realization is both controllable and observable and it is minimal because the system is modeled with the minimum number of states.

Figure 10 shows a dynamic network with two state variables $y_{1(k)}$ and $y_{2(k)}$ but only one measured variable $y_{1(k)}$. In this case, the cost function to be minimized must be defined in terms of the measured variable $y_{1(k)}$ only, as it is shown in equation (15).



$$y_{k+1} = f(y_k, u_k) \quad y_k = \begin{bmatrix} y_{1(k)} \\ y_{2(k)} \end{bmatrix} \quad u_k = \begin{bmatrix} u_{1(k)} \\ u_{2(k)} \end{bmatrix}$$

$$J = 0.5 (y_{1(1)} - y_{1(1)}^*)^2 + 0.5 (y_{1(2)} - y_{1(2)}^*)^2 + 0.5 (y_{1(3)} - y_{1(3)}^*)^2 + \dots + 0.5 (y_{1(N)} - y_{1(N)}^*)^2 \quad (15)$$

This cost function can be defined in terms of the complete state vector $y_k = [y_{1(k)} \ y_{2(k)}]^T$ considering the weighting matrix Q as it is shown in equation (16).

$$J = 0.5(y_1 - y_1^*)^T Q (y_1 - y_1^*) + 0.5(y_2 - y_2^*)^T Q (y_2 - y_2^*) + \dots + 0.5(y_N - y_N^*)^T Q (y_N - y_N^*) \quad (16)$$

$$J = \frac{1}{2} \sum_{k=1}^N (y_k - y_k^*)^T Q (y_k - y_k^*)$$

where matrix Q is equal to:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

The total partial derivatives of the cost function J respect to weighting coefficients v and w are given by the following equations:

$$\frac{\partial J}{\partial v} = (y_1 - y_1^*)^T Q \frac{\partial y_1}{\partial v} + (y_2 - y_2^*)^T Q \frac{\partial y_2}{\partial v} \dots \dots + (y_N - y_N^*)^T Q \frac{\partial y_N}{\partial v} \quad (17)$$

$$\frac{\partial J}{\partial w} = (y_1 - y_1^*)^T Q \frac{\partial y_1}{\partial w} + (y_2 - y_2^*)^T Q \frac{\partial y_2}{\partial w} \dots \dots + (y_N - y_N^*)^T Q \frac{\partial y_N}{\partial w} \quad (18)$$

The total partial derivatives of the state vector respect to the weighting coefficients v and w :

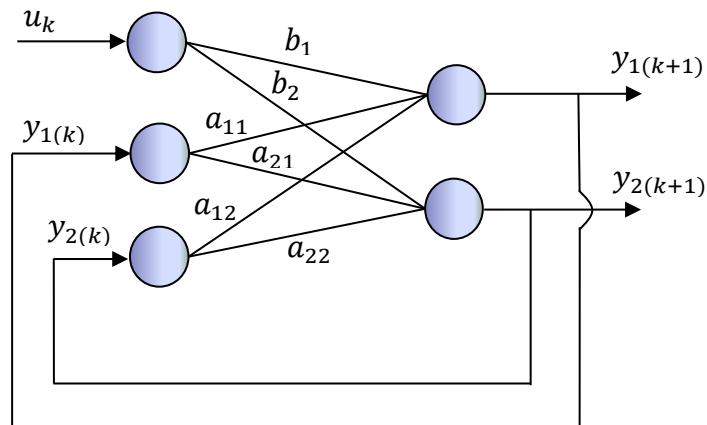
$$\begin{aligned} \frac{\partial y_1}{\partial v}, \frac{\partial y_2}{\partial v}, \dots, \frac{\partial y_N}{\partial v} \\ \frac{\partial y_1}{\partial w}, \frac{\partial y_2}{\partial w}, \dots, \frac{\partial y_N}{\partial w} \end{aligned}$$

are computed using the recursive expression of equation (10).

Linear Systems Parameter Identification

For some systems, the neural network training process can also be applied to identify the parameters of the state-space equation of the system as long as the equations can be represented by a neural network. In the case of linear systems this representation is straightforward considering a neural network with linear neurons.

$$\begin{bmatrix} y_{1(k+1)} \\ y_{2(k+1)} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} y_{1(k)} \\ y_{2(k)} \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} u_{(k)}$$



$$y_{k+1} = Ay_k + Bu_k \quad y_k = \begin{bmatrix} y_{1(k)} \\ y_{2(k)} \end{bmatrix}$$

The weighting coefficients a_{ij} and b_i can be determined by network training.

Programas de Matlab

Modelamiento con Red Estática

MotorNeuroEstatico.m
MotorNeuroEstatico1.m

Modelamiento con Red Dinámica

Sistema de dos variables
DynamocBPModelamiento2v.m

Sistema de tres variables
DynamicBPModelamiento3v.m

Red neuronal de dos capas intermedias
DynamicBPModelamientoDosIntermedias.m

Sistema Lineal – Identificación de Parámetros
DynamicBPLinealModel2.m